

Investigación #2: Redes programables

Universidad de Costa Rica
Escuela de Ciencias de la Computación e Informática

CI-0121 Redes de comunicación de datos
I Semestre – 2026

Estudiantes:

Anferhny Berrocal Rojas (C11056)

Adrián Arias Vargas (C30749)

Introducción

Por mucho tiempo, los routers y switches han funcionado como cajas cerradas: el fabricante define qué protocolos pueden entender y cómo procesan los paquetes, mientras que el operador de red solo puede modificar ciertos parámetros dentro de esas limitaciones. Aunque este modelo ha permitido construir redes eficientes, también ha hecho que la innovación dependa en gran parte del hardware disponible y de las decisiones del fabricante.

En este informe se explica qué es P4 (Programming Protocol-Independent Packet Processors), un lenguaje que busca cambiar este modelo al permitir que el programador defina cómo se procesan los paquetes en el plano de datos. Para esto, se revisa la evolución del plano de datos fijo hacia SDN y los planos de datos programables, la arquitectura PISA, el rol de P4Runtime y algunos casos de uso reales, como telemetría, balanceo de carga, seguridad y aplicaciones en redes de producción.

Evolución del plano de datos

Limitaciones del modelo tradicional

En una red tradicional, el mismo dispositivo de red, sea switch o router, decide por sí mismo tanto las rutas como el plano de datos. El problema con este modelo es que el plano de datos es fijado por el fabricante desde el momento en el que se diseña el chip, es decir, solo entiende los protocolos para los que fue construido, y si se quiere soportar un protocolo nuevo hay que esperar a que se diseñe un chip nuevo con ese protocolo en cuenta. Esto hace que la innovación en redes sea lenta y cara, porque depende del actuar de los fabricantes de hardware.

Surgimiento del plano de datos programable

Las redes definidas por software o SDN surgen como un intento de resolver las limitaciones del modelo tradicional, separando los planos de control y de datos, usando un controlador centralizado que dirige a los switches mediante una interfaz

estandarizada. El protocolo más conocido para esto fue OpenFlow, pero este sigue trabajando sobre un conjunto de tamaño fijo de campos de encabezado ya estandarizados, por lo que si se quiere agregar nuevas reglas hay que esperar una nueva versión del protocolo. Esto se denota en el propio artículo que propone P4, donde se menciona que el conjunto de campos que soporta OpenFlow pasó de 12 a 41 en pocos años, y aun así no contaba con la flexibilidad necesaria para introducir encabezados nuevos (Bosshart et al., 2014). Producto de esta frustración nace la idea de un plano de datos realmente programable, donde el operador describe en un lenguaje de alto nivel cómo quiere que se procese cada paquete, y esa descripción se compila para distintos tipos de hardware.

SDN clásica (OpenFlow) frente a P4

La diferencia entre OpenFlow y P4 no es necesariamente que uno use un controlador centralizado y el otro no, sino más bien el nivel de abstracción en el que trabaja el plano de datos. Un survey de ACM Computing Surveys explica esta evolución describiendo a OpenFlow como una API limitada a un conjunto fijo de mensajes y campos, mientras que P4 es un lenguaje con un dominio específico que permite describir arquitecturas completas de procesamiento de paquetes y llevarlas a distintos tipos de hardware (Hauser et al., 2022). En la práctica, con OpenFlow uno le dice al switch "aplica X regla sobre estos campos que ya existen", mientras que con P4 se le puede decir "así se ven los encabezados que quiero reconocer, y así se deben de procesar", incluso si son protocolos que no existían al momento de fabricación el switch. Por esto principalmente, en la comunidad de redes se dice que OpenFlow más o menos dejó de avanzar y que el desarrollo se movilizó hacia P4 (Hauser et al., 2022).

Lenguaje P4

Historia y motivación

P4 se propuso formalmente en 2014, en un paper publicado en ACM SIGCOMM Computer Communication Review llamado "P4: Programming Protocol-Independent Packet Processors", escrito por un grupo de autores

compuesto por Barefoot Networks, Intel, Stanford, Princeton, Google y Microsoft Research (Bosshart et al., 2014). En este paper se plantean tres metas de diseño que siguen siendo la base del lenguaje a día de hoy: reconfigurabilidad en campo, poder cambiar el procesamiento de paquetes por parte de un switch ya desplegado; independencia de protocolo, no asumir de antemano qué encabezados van a existir e independencia de target, que el mismo programa pueda ser compilado para un hardware distinto, sin importar sus características específicas. Vale la pena mencionar que P4 no busca reemplazar a OpenFlow como protocolo para configurar el plano controlador, de hecho los mismos autores lo presentaron como una propuesta de cómo podría evolucionar OpenFlow y, normalmente, P4 se usa junto con interfaces de control como P4Runtime (Bosshart et al., 2014).

Arquitectura de un programa P4

Un programa P4 describe el comportamiento de un switch usando principalmente cuatro etapas de procesamiento. Como describen Gao et al. (2021), esas etapas son el parser, las tablas match-action, los flujos de control y el deparser:

- Parser: es la primera etapa del procesamiento y se encarga de recibir el paquete tal cual llega (cadena de bits) y extraer sus encabezados de acuerdo con el formato definido por el programador, y darles una forma estructurada.
- Tablas match-action: son el componente central del procesamiento. Cada tabla compara campos del paquete contra un conjunto de entradas y ejecuta la acción de la entrada que coincide. Las entradas de estas tablas normalmente son instaladas y actualizadas por el plano de control en tiempo de ejecución.
- Flujos de control: también conocidos como bloques de control, aquí el programador organiza el orden en que se aplican las tablas, estableciendo condiciones para decidir qué tablas aplicar según el tipo de paquete, el resultado de una búsqueda anterior o el estado de ciertos encabezados.
- Deparser: es la parte final del procesamiento y su función es reconstruir el paquete antes de enviarlo por el puerto de salida. Para esto, vuelve a emitir

los encabezados que siguen siendo válidos después del procesamiento, incluyendo aquellos que pudieron haber sido modificados por las acciones del programa .

Esta forma de separar el procesamiento en etapas es justo lo que logra que P4 sea independiente de protocolo, el lenguaje en sí mismo no sabe qué es un encabezado IPv4 o Ethernet, eso lo define el programador como si fueran estructuras de datos estándar (Bosshart et al., 2014).

El modelo de ejecución PISA

La arquitectura PISA (Protocol-Independent Switch Architecture) es uno de los modelos más importantes para entender el funcionamiento de los switches programables con P4 . Esta describe un pipeline genérico que está compuesto por tres bloques programables, el parser, un conjunto de unidades match-action (MAU) y el deparser (P4.org, 2019). En un switch real basado en PISA, el parser convierte el paquete en un vector de encabezados que se guarda en una memoria intermedia. Ese vector pasa por varias etapas MAU, cada una con su propia memoria y sus unidades de cálculo, que comparan campos del PHV contra tablas y ejecutan acciones que pueden modificar el PHV o manipular registros que se mantienen entre paquetes. Al final, el deparser vuelve a armar el paquete y lo manda por el puerto de salida (P4.org, 2019). Este modelo ayuda a entender por qué P4 es independiente de protocolo, el parser, las tablas y el deparser son completamente programables, y PISA solo define la forma general del pipeline, no lo que hay dentro (P4.org, 2019).

Tipos de targets

Un mismo programa P4 se puede compilar, con algunos ajustes, para distintos tipos de hardware o software:

- BMv2 (Behavioral Model versión 2): es el switch de software de referencia usado por la comunidad de P4 Language Consortium. Se utiliza sobre todo para pruebas, prototipos y entornos académicos porque no está pensado para rendimiento alto.

- Tofino: es la familia de ASICs programables que desarrolló originalmente Barefoot Networks. Es el target comercial de alto rendimiento más conocido para P4, capaz de procesar tráfico a velocidades de hasta 12.8 Tbps manteniendo el pipeline programable (Fierce Network, 2018).
- FPGAs: también se ha explorado compilar programas P4 hacia FPGAs, aprovechando que se pueden reconfigurar, aunque con retos propios de diseño de hardware que no se dan en los ASICs de propósito específico.

P4Runtime y el plano de control

El rol del plano de control

Un programa P4 define cómo se comporta el plano de datos, pero no decide por sí solo qué entradas concretas debe tener cada tabla en un momento dado, por ejemplo qué rutas IP existen o qué reglas de seguridad aplicar. Eso sigue siendo trabajo del plano de control, que puede ser un controlador SDN remoto, un proceso local en el mismo dispositivo, o una mezcla de ambos. Al compilar un programa P4 para un target específico, se genera la configuración que ejecutará el switch y un archivo de metadatos llamado P4Info. Este archivo describe las tablas, acciones y demás objetos definidos en el programa, permitiendo que el plano de control manipule el switch sin depender directamente del código P4 original (P4.org, 2019).

Protocolo P4Runtime

P4Runtime es la especificación de la API del plano de control para dispositivos definidos o descritos mediante P4. Esta API permite que un controlador administre los elementos del plano de datos de un switch programable, como tablas, contadores, registros y otros objetos definidos en el programa P4. El controlador primero instala la configuración del pipeline junto con el archivo P4Info correspondiente, el cual describe las entidades disponibles y los identificadores que deben usarse para manipularlas. La comunicación entre el controlador y el switch se hace mediante gRPC, usando mensajes definidos en Protocol Buffers. El servidor de P4Runtime escucha por defecto en el puerto TCP 9559, que fue asignado

específicamente para esto. Usar gRPC tiene sus ventajas frente a protocolos más viejos, como poder tener streams bidireccionales para funciones como la elección de un controlador principal cuando hay varios controladores por razones de alta disponibilidad (P4.org API Working Group, 2024).

Comparación con OpenFlow

P4Runtime y OpenFlow buscan más o menos lo mismo, que un controlador externo pueda programar el comportamiento de un switch. La diferencia está en qué tan general es cada uno. OpenFlow por su lado ya trae definida la semántica de los campos que se pueden usar para hacer match como parte fija del protocolo. P4Runtime en cambio no sabe nada de eso por sí mismo, todo depende del P4Info que se generó a partir del programa P4 que uno haya escrito, lo cual lo hace mucho más independiente de protocolo. Esto podría sonar como que cuesta algo en rendimiento, porque cada instalación necesita resolver identificadores en vez de usar campos fijos directamente, pero en la práctica las diferencias de desempeño no son muy grandes y P4 puede llegar a igualar la velocidad de protocolos de campos fijos como OpenFlow (Hauser et al., 2022; P4.org API Working Group, 2024).

Casos de uso relevantes

In-Band Network Telemetry (INT)

INT es probablemente la aplicación más conocida de P4 fuera del enrutamiento básico. La idea de esta técnica es que cada switch por el que pasa un paquete inserte, dentro del mismo paquete, información sobre su propio estado interno, como ocupación de colas, marca de tiempo o uso del enlace de salida, para que al final un nodo (el sink) pueda armar una vista detallada del camino que siguió el paquete sin tener que consultarle a cada switch por separado (CodiLime, 2023).

Esto solo es posible gracias a que el parser y el deparser de P4 son programables. Un ejemplo concreto de esto es BurstRadar, un sistema que corre sobre switches Tofino programados en P4 para detectar microbursts de tráfico directamente en el plano de datos. Se evaluó usando patrones de tráfico tomados de

la red de producción de Facebook, y logró reducir hasta diez veces la sobrecarga de recolección de datos comparado con soluciones anteriores (Joshi et al., 2018).

Load balancing programable

El hecho de que P4 permita guardar estado por flujo directamente en registros del switch abrió la puerta a balanceadores de carga que funcionan completamente en el plano de datos, sin necesitar un balanceador de software externo revisando cada paquete. Un ejemplo es HULA, presentado en el Symposium on SDN Research (SOSR) 2016 por investigadores de Princeton y VMware. Es un algoritmo de balanceo sensible a la congestión, donde cada switch solo necesita conocer cuál es el mejor siguiente salto hacia un destino, en lugar de guardar el estado de congestión de todos los caminos posibles como hacían los esquemas anteriores. Los mismos autores programaron HULA en P4 para mostrar que podía correr en switches programables sin necesitar hardware diseñado específicamente para ese algoritmo (Katta et al., 2016).

Funciones de seguridad en el plano de datos

La capacidad de P4 para mantener registros con estado y tomar decisiones a velocidad de línea también se ha usado para implementar defensas contra ataques directamente en los switches, sin depender de enviar todo el tráfico a un controlador externo. Esto es importante porque remitir cada paquete al controlador aumentaría la latencia y la carga del sistema, especialmente ante ataques de gran volumen como un DDoS. Un survey sobre este tema documenta varios mecanismos de este tipo, como el filtrado de paquetes SIP maliciosos en ataques de denegación de servicio telefónica mediante registros que cuentan mensajes sin su respuesta correspondiente, o la mitigación de inundaciones SYN usando registros que llevan la cuenta de intentos de conexión y descartan el tráfico una vez que se supera un umbral configurable (Gao et al., 2021). En este mismo survey se menciona que el plano de datos programable con P4 puede mitigar ataques de gran escala con poca sobrecarga, aunque también presenta limitaciones, como la memoria reducida del switch y la dificultad de realizar operaciones complejas, por ejemplo divisiones (Gao et al., 2021).

AT&T y telemetría en banda

Más allá de los laboratorios y papers académicos, ya hay casos documentados de P4 funcionando en redes de producción reales. Uno de ellos es el de AT&T, reportado por Fierce Network, donde se instalaron switches de caja blanca basados en el chip Tofino, corriendo el sistema FlexSwitch de SnapRoute, dentro de la red MPLS de la compañía, con equipos puestos en San Francisco y en Washington D.C. (Fierce Network, 2018). Según ese mismo reporte, AT&T usó la programabilidad del chip para mejorar la visibilidad de su tráfico mediante telemetría en banda, logrando ver el comportamiento del tráfico a nivel nacional a partir de esos dos puntos instrumentados (Fierce Network, 2018). Este caso ayuda a ver que todo lo que se ha abarcado en este informe no es solo teoría, sino que ya se está usando en una red de un operador de telecomunicaciones grande.

Conclusiones

Después de realizar esta investigación, queda claro que P4 no es simplemente “una versión más flexible de OpenFlow”, sino una propuesta con un nivel de abstracción distinto. En lugar de que el switch venga limitado a los protocolos fijados de fábrica, P4 permite que el programador describa qué encabezados reconocer y cómo procesar los paquetes, manteniendo la idea de SDN de separar el plano de control del plano de datos.

El modelo PISA, con parser, match-action y deparser, ayuda a entender cómo se procesan los paquetes en dispositivos programables. Esta misma estructura puede adaptarse a distintos targets, desde BMv2 hasta ASICs como Tofino. Además, P4Runtime permite controlar esa flexibilidad desde afuera, sin depender directamente del código P4 original.

Los casos revisados muestran que P4 ya no es solo teoría. La telemetría en banda, el balanceo de carga, la seguridad en el plano de datos y el caso de AT&T muestran que ya se usa para resolver problemas reales en redes modernas.

Referencias

- Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G., & Walker, D. (2014). P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, 44(3), 87–95. <https://doi.org/10.1145/2656877.2656890>
- CodiLime. (2023, enero 30). In-band network telemetry — what is it all about? CodiLime Blog. <https://codilime.com/blog/in-band-network-telemetry-what-is-it-all-about/>
- Fierce Network. (2018). Barefoot Networks gears up for AI, 5G with P4 and its Tofino chip. <https://www.fierce-network.com/telecom/barefoot-networks-gears-up-for-ai-5g-p4-and-its-tofino-chip>
- Gao, S., Peng, Z., Xiao, B., Hu, A., Song, Y., & Ren, K. (2021). A review of P4 programmable data planes for network security. *Mobile Information Systems*, 2021, Artículo 1257046. <https://doi.org/10.1155/2021/1257046>
- Hauser, F., Häberle, M., Merling, D., Lindner, S., Gurevich, V., Zeiger, F., Frank, R., & Menth, M. (2022). Advancing SDN from OpenFlow to P4: A survey. *ACM Computing Surveys*, 54(9), Artículo 196. <https://doi.org/10.1145/3556973>
- Joshi, R., Qu, T., Chan, M. C., Leong, B., & Loo, B. T. (2018). BurstRadar: Practical real-time microburst monitoring for datacenter networks. *Proceedings of the 9th ACM Asia-Pacific Workshop on Systems (APSys '18)*, Artículo 8. <https://doi.org/10.1145/3265723.3265731>
- Katta, N., Hira, M., Kim, C., Sivaraman, A., & Rexford, J. (2016). HULA: Scalable load balancing using programmable data planes. *Proceedings of the Symposium on SDN Research (SOSR '16)*, 1–12. <https://doi.org/10.1145/2890955.2890968>

P4.org. (2019). P4 language overview. CS344, Stanford University.

<https://cs344-stanford.github.io/lectures/Lecture-2-P4-tutorial.pdf>

P4.org. (s. f.). P4Runtime specification. P4 Language Consortium.

<https://p4.org/wp-content/uploads/sites/53/2024/10/P4Runtime-Spec-v1.4.1.pdf>